

DH2323 Computer Graphics and Interaction

Comparison between geometric and physics based deformation models

Vanessa Neef (neef@kth.se), Elín Fririka Hermannsdóttir (efhe@kth.se)

KTH Royal Institute of Technology, Stockholm, Sweden

May 25, 2026

Abstract—This paper presents the implementation and comparative evaluation of two object deformation methods, Free-Form Deformation (FFD) and Position-Based Dynamics (PBD), within an immersive virtual reality environment. Both methods were implemented in Unity and deployed on a Meta Quest 3 headset. A qualitative user study involving four participants was conducted to assess the influence of each method on perceived visual realism, physical plausibility and overall interaction quality. Participants favoured the geometry-based FFD approach across all areas. This makes it better suited to soft, compliant materials, with participants finding it more predictable and satisfying to interact with. While the physics-based PBD implementation is theoretically well-suited to soft-body simulation, it did not achieve the desired level of responsiveness. This limited the comparability of the two conditions. This study should be viewed as a pilot investigation that highlights methodological considerations for future research rather than drawing generalisable conclusions. The findings suggest that FFD may offer a more accessible route to achieving convincing soft-body interaction in mobile VR, while also raising questions for more controlled follow-up studies.

Keywords: *Object Deformation, Free-Form Deformation, Position Based Dynamics, Virtual Reality, Computer Graphics*

1. INTRODUCTION

Extended reality (XR) applications rely heavily on the convincing graphical representation and interaction of virtual objects [10, 12]. Real-time physics simulations are considered essential for immersive XR environments because they enhance realism, user experience, and the overall sense of presence within a virtual scene [10]. Realistic object behaviour contributes directly to the perceived success and usability of XR systems [10]. Different deformation and simulation techniques therefore play an important role across a variety

of XR applications, including virtual clothing fitting, medical training, educational environments, and interactive art installations [7]. A defining characteristic of XR environments is the close integration of physical space and virtual content. To maintain immersion, virtual objects must react plausibly to user interactions and environmental changes. This makes object deformation a particularly important aspect of computer graphics for XR. Deformable object modelling has been studied extensively for decades and has been applied to the animation of clothing, facial expressions, and other

dynamic objects [4]. Most deformation approaches are based on physical principles that compute object behaviour through simulations of material properties and forces. However, non-physical approaches also exist, relying purely on geometric techniques. These methods are often computationally efficient and include approaches such as splines, patches, and free-form deformation [4, 3].

In interactive environments, objects can undergo different forms of transformation. Besides movement-based animations such as keyframe or motion-path animation, changes in an object’s shape are commonly referred to as morphing or deformation [11]. Deformation specifically describes the process of altering an object’s shape over time by applying mathematical functions to its vertices, thereby repositioning them to create a modified geometry [11]. Depending on the desired visual effect and interaction requirements, selecting an appropriate deformation model can be challenging. Different mathematical approaches offer varying trade-offs between realism, computational efficiency, controllability, and responsiveness, all of which are critical factors in real-time XR applications [1, 11].

This project involves implementing and comparing two well-established deformation methods, Free-Form Deformation (FFD) and Position-Based Dynamics (PBD), within an interactive virtual reality environment. The aim of the study is to find out how well these two approaches, one geometric and one physics-based, work for immersive XR applications. The sections that follow present the theoretical foundations of both methods, their practical implementation in Unity, and their evaluation through a qualitative user study.

2. THEORETICAL FRAMEWORK

This section presents the theoretical foundation on which the implementation and evaluation of this project are based. It introduces Free-Form Deformation (FFD) and Position-

Based Dynamics (PBD) as the two graphical methods used in the project and provides an overview of their mathematical foundations.

2.1 Free-Form Deformation (FFD)

Free-Form Deformation (FFD) is a geometric technique for deforming solid 3D models by embedding them within a flexible lattice of control points. Originally introduced by Sederberg and Parry [9], FFD enables smooth, continuous deformation of arbitrary 3D geometry without requiring direct manipulation of individual mesh vertices. Instead, the user manipulates a sparse set of control points arranged in a parallelepiped, a three-dimensional grid whose displacement propagates to the embedded mesh through trivariate Bernstein polynomial blending functions, a technique later extended to dynamic settings by embedding Lagrangian mechanics directly into the FFD framework Faloutsos, Van De Panne, and Terzopoulos [3].

FFD was selected for this study as the representative geometric deformation method due to its mathematical rigour, its widespread use in both academic research and production 3D software, and its ability to produce smooth, globally coherent deformations from a small number of user-controlled parameters.

2.2 Mathematical Foundations of FFD

This section presents the mathematical foundations of Free-Form Deformation (FFD) based on the work of Sederberg and Parry [9]. It describes the spatial parameterisation of embedded vertices, the construction of the control point lattice, and the deformation model used to compute vertex positions based on control point displacements.

2.2.1 Local coordinate system

FFD begins by establishing a local coordinate frame aligned with the bounding box of the mesh. Three orthogonal axes are defined along

the spatial extents of the object:

$$\begin{aligned} S &= (x_{\max} - x_{\min}, 0, 0) \\ T &= (0, y_{\max} - y_{\min}, 0) \\ U &= (0, 0, z_{\max} - z_{\min}) \end{aligned}$$

The origin X of this frame is placed at the minimum corner of the bounding box. Together, S , T , and U define a local space in which every vertex of the mesh can be expressed as a normalised coordinate between 0 and 1 along each axis.

2.2.2 Vertex Parameterization

Each vertex p of the original mesh is mapped into this local space, producing a normalised coordinate triple $(s, t, u) \in [0, 1]^3$:

$$\begin{aligned} s &= (p - X_0) \cdot \tilde{S} \\ t &= (p - X_0) \cdot \tilde{T} \\ u &= (p - X_0) \cdot \tilde{U} \end{aligned}$$

where $\tilde{S}$, $\tilde{T}$, $\tilde{U}$ are the unit vectors along each axis. This parameterization captures where each vertex sits within the control point lattice and is computed once at initialisation. Because the local coordinates of a vertex do not change when control points move, they can be stored and reused for every subsequent deformation update.

2.2.3 Control Point Lattice

A three-dimensional grid of control points P_{ijk} is placed uniformly within the local frame. The grid has dimensions $(L + 1) \times (M + 1) \times (N + 1)$, where L , M , and N are integers specifying the resolution along S , T , and U respectively. The rest position of each control point is

$$P_{ijk} = X_0 + \frac{i}{L}S + \frac{j}{M}T + \frac{k}{N}U.$$

for $i \in [0, L]$, $j \in [0, M]$, $k \in [0, N]$. When the user displaces a control point from its rest position, the surrounding mesh deforms in response.

2.2.4 The FFD Deformation Formula

The deformed position p' of a vertex is computed as a weighted sum over all control points in the lattice. The weights are given by Bernstein basis polynomials, which produce smooth, localised influence that decays with parametric distance. The formula, as given by Sederberg and Parry [9], is:

$$p'(s, t, u) = \sum_{i=0}^L \sum_{j=0}^M \sum_{k=0}^N B_i^L(s) B_j^M(t) B_k^N(u) P_{ijk},$$

where $B_i^n(x)$ is the Bernstein basis polynomial of degree n :

$$B_i^n(x) = \binom{n}{i} x^i (1-x)^{n-i},$$

and $\binom{n}{i}$ is the binomial coefficient.

The key property of this formulation is that the Bernstein polynomials always sum to 1, which means that when no control point has been moved, every vertex is reconstructed exactly at its original position. When a control point is displaced, its influence on a given vertex is proportional to how close that vertex is to the control point in parametric space, vertices nearby are strongly affected, while vertices further away are affected less. This formulation operates purely geometrically; the control points carry no physical properties. Faloutsos, Van De Panne, and Terzopoulos [3] later extended this by assigning mass and elasticity to the embedded object, enabling the lattice to evolve according to Lagrangian dynamics instead of being manipulated manually.

2.3 Position-Based Dynamics (PBD)

Position-Based Dynamics (PBD), introduced by Müller et al. [8], is a widely used method for real-time simulation of deformable bodies in games and interactive applications, valued particularly for its simplicity and robustness. Unlike traditional force-based approaches, which compute forces first and derive motion through Newton's second law,

PBD skips force calculation entirely and works directly on vertex positions [8, 2]. This contrasts with earlier physics-based approaches [3], which applied Lagrangian dynamics directly to FFD lattices rather than skipping the force calculation step.

The simulation proceeds in four stages per time step: (1) external forces such as gravity update velocities; (2) predicted positions are computed for each vertex; (3) an iterative constraint solver adjusts those positions to satisfy physical rules such as distance preservation and volume conservation; and (4) velocities are updated based on how much positions actually moved. This makes PBD faster, more stable, and easier to implement than force-based methods like mass-spring systems or FEM, while still producing visually plausible behaviour [8, 10].

A key advantage is direct control over integration, overshooting and energy-gain problems typical of explicit force-based schemes are avoided, and vertex positions can be manipulated at any point during simulation [8, 10].

PBD was selected as the physics-based condition in this study because it is the deformation model underlying Unity’s built-in soft-body and cloth systems, making it directly relevant to real-time VR development. Sung et al. [10] demonstrated a GPU-accelerated PBD framework deployed within Unity for XR applications, combining a PBD solver with sophisticated collision detection to achieve real-time performance with multiple deformable objects. Their work confirms PBD as the current standard for physics-based deformation in interactive XR environments, providing the practical and academic context for its use as the physics-based condition in this comparison [10].

2.4 Mathematical Foundations of PBD

The PBD simulation loop proceeds in three phases per time step, governed by the following update equations [8, 10, 2].

Prediction step. At each time step, external forces are used to estimate a predicted position $\tilde{x}_i$ for each vertex:

$$\tilde{x}_i = x_i + \Delta t \cdot v_i + \Delta t^2 \cdot M^{-1} f_{\text{ext}}$$

where x_i is the current position, v_i the current velocity, M the mass matrix, and f_{ext} the external force vector. These predictions may violate the object’s physical constraints.

2.4.1 Constraint Projection

The core of PBD is an iterative solver that corrects each predicted position to satisfy a constraint $C_j(x_1, \dots, x_n) = 0$. For each constraint, a correction Δx_i is computed as:

$$\Delta x_i = -\frac{w_i C_j(\tilde{x}_1, \dots, \tilde{x}_n)}{\sum_k w_k \|\nabla_{x_k} C_j\|^2} \nabla_{x_i} C_j$$

where $w_i = 1/m_i$ is the inverse mass of vertex i . This is applied iteratively for a fixed number of solver passes, progressively reducing constraint violations.

2.4.2 Distance Constraints

The most fundamental constraint for soft-body deformation preserves the rest length d between two connected vertices p_1 and p_2 :

$$C(p_1, p_2) = \|p_1 - p_2\| - d = 0$$

The position corrections, weighted by inverse mass, are:

$$\Delta \mathbf{p}_1 = -\frac{w_1}{w_1 + w_2} (\| \mathbf{p}_1 - \mathbf{p}_2 \| - d) \frac{\mathbf{p}_1 - \mathbf{p}_2}{\| \mathbf{p}_1 - \mathbf{p}_2 \|} \quad (1)$$

$$\Delta \mathbf{p}_2 = +\frac{w_2}{w_1 + w_2} (\| \mathbf{p}_1 - \mathbf{p}_2 \| - d) \frac{\mathbf{p}_1 - \mathbf{p}_2}{\| \mathbf{p}_1 - \mathbf{p}_2 \|} \quad (2)$$

A stiffness parameter $k \in [0, 1]$ scales the correction, allowing the material to be tuned from

fully elastic to rigid [8, 10, 2]. After all constraints are resolved, vertex velocities are updated as:

$$v_i = \frac{\tilde{x}_i - x_i}{\Delta t}$$

3. IMPLEMENTATION

Both deformation methods were implemented in Unity (C#) and tested on a sphere mesh to allow direct visual comparison under consistent conditions. The sphere served as a controlled test object, providing a simple and symmetric geometry that makes deformation behaviour easy to observe and evaluate. The FFD implementation deforms the sphere by manipulating a lattice of interactive control points, while the PBD implementation simulates the sphere as a volumetric soft body driven by physical constraints.

3.1 Free-Form Deformation Implementation

The FFD implementation follows the Sederberg and Parry [9] formulation directly. On startup, a bounding box is computed around the sphere mesh and a $3 \times 3 \times 3$ lattice of control points is placed at evenly spaced positions across the volume, with each point instantiated as a small sphere prefab in the scene. For each mesh vertex, its local coordinates $(s, t, u) \in [0, 1]^3$ are computed relative to the bounding box axes, and the Bernstein basis weights for each axis are pre-baked and stored in a `Vector3Param` object. This pre-computation means that during deformation, no trigonometric or polynomial evaluation is needed per frame [9].

When deformation is enabled, the mesh is updated at a fixed interval (every 50 ms by default) via `FixedUpdate`. Each vertex position is recomputed using the trivariate Bernstein sum:

$$\mathbf{x}' = \sum_{i=0}^L \sum_{j=0}^M \sum_{k=0}^N B_{i,L}(s) B_{j,M}(t) B_{k,N}(u) \mathbf{P}_{i,j,k} \quad (3)$$

where $\mathbf{P}_{i,j,k}$ are the current world positions of the control points. After all vertices are updated, bounds and normals are recalculated. The user interacts with the deformation by clicking and dragging the control point spheres in the scene, which is handled by `ControlPointDragger` using Unity's `OnMouseDown` callback projected into world space.

3.1.1 Modifications for VR

The original FFD implementation used Unity's `OnMouseDown` callback for control point interaction, which relies on mouse input and is incompatible with VR. To support interaction in the Meta Quest 3 environment, the `ControlPointDragger` component was disabled on the `ControlFreeFab` prefab to preserve the original editor workflow, and an `XRInputHandler` script was added alongside it. This script registers listeners on an `XRSimpleInteractable` component for the `selectEntered` and `selectExited` events, which are fired by the XR Interaction Toolkit when the controller ray selects an object. On grab, the controller transform is stored and the offset between the control point and the controller position is recorded. During the grab, the control point follows the controller position plus the original offset each frame in `Update()`, replicating the drag behaviour of the original mouse-based interaction. A `SphereCollider` was added to the prefab to serve as the raycast target for the XR ray interactor. Additionally, the XR Origin was replaced with the Starter Assets XR Origin (XR Rig) prefab, which includes pre-configured Ray Interactors on both controllers, enabling ray-based selection out of the box.

3.2 Position-Based Dynamics

The PBD implementation is based on the open-source Unity project by Sung et al. [10], accompanying their paper "Real-Time Physics Simulation Method for XR Application" [10]. The original project provides both CPU and GPU-based PBD solvers for deformable objects in Unity. The CPU-based solver was selected, as the Meta Quest 3's mobile GPU does not support the compute shader pipeline required by the GPU version.

3.2.1 Mesh and Constraints

The solver operates on a volumetric tetrahedral mesh generated using TetGen, stored across five data files defining node positions, tetrahedral elements, surface triangles, distance constraints, and bending constraints. These are loaded at runtime by the `LoadTetModel` class, which constructs the constraint lists used by the solver. Each frame, the `CPUPBD MonoBehaviour` applies gravity, integrates positions explicitly, iteratively satisfies distance, bending, and volume constraints, handles floor collision, and updates node positions. The sphere is rendered procedurally via `Graphics.DrawProcedural`, since the original repository included no rendering shader, a custom HLSL shader was written to render the surface from the node and triangle buffers.

3.2.2 Modification for Android and VR

The original file loading used `System.IO.StreamReader` with `Application.dataPath`, which is inaccessible on Android. The mesh files were moved to Unity's `StreamingAssets` folder and loading was replaced with a coroutine using `UnityWebRequest` to resolve Android's `jar://` URI scheme, writing files to `Application.temporaryCachePath` before parsing. A `simReady` flag was added to prevent the solver running before loading completes. The floor collision response was modified to zero only the vertical

velocity component rather than the full vector, allowing interaction forces to persist through the collision step. Finally, a public `ApplyForceAtPoint()` method was added to inject velocity impulses into nodes within a given world-space radius.

3.2.3 VR Interaction

Interaction follows the same pattern as the FFD condition. An `XRSimpleInteractable` and `SphereCollider` were added to the sphere and the collider serving as the raycast target since the procedural mesh has none by default. When the user points at the sphere and presses select, the `PBDVRInteraction` script computes the controller's frame-to-frame displacement and passes it as a velocity impulse to `ApplyForceAtPoint()` each frame, deforming nodes near the interaction point.

4. EVALUATION METHOD

This evaluation investigates how different object transformation methods influence user perception in an interactive VR environment and compares FFD with PBD.

The evaluation focuses on three key aspects: visual realism, physical plausibility, and overall user perception. Visual realism refers to how natural and visually convincing the deformations appear, while physical plausibility describes whether the observed behaviour aligns with real-world physical expectations. User perception captures the subjective experience of participants when interacting with the different transformation methods.

To assess these aspects, participants interact with objects deformed using each method and provide feedback based on their experience. The goal is to understand how the underlying deformation approach influences perceived quality and realism in VR-based object manipulation.

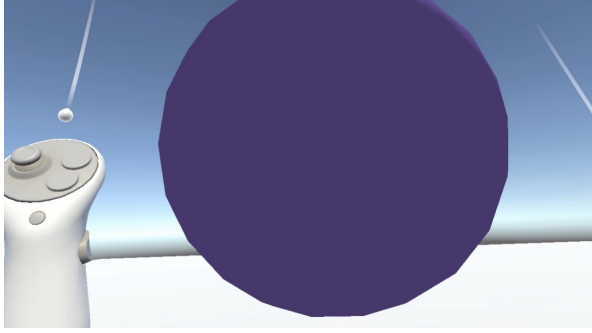


Figure 1: PBD based scene

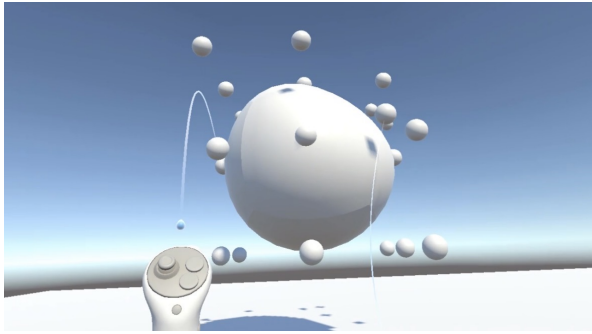


Figure 2: FFD based scene

4.1 Participants

Four participants aged 21-24 were recruited through convenience sampling. All participants were students in Interactive Media Technology at KTH Royal Institute of Technology.

4.2 Test Environment and Apparatus

The evaluation was conducted in an immersive VR environment developed in Unity 6.4 and experienced using a Meta Quest 3 headset. Two separate scenes were created, one for each graphical method. The physics-based scene consisted of a plane and a sphere. The geometry-based scene also contained a plane and a sphere, together with several smaller spheres used to handle the interaction.

4.3 Procedure

The study used a within-subject design, meaning that all participants experienced both conditions. To minimise order effects, half of the participants started with Method A and the other half with Method B. Before the evaluation began, participants were informed about

the purpose of the study, the data being collected and stored, the potential risk of motion sickness, and their right to withdraw from the experiment at any time. Participants then provided informed consent. To assess motion sickness, the Virtual Reality Sickness Questionnaire (VRSQ) [6] was used instead of the more established Simulator Sickness Questionnaire (SSQ) [5]. The VRSQ was selected because the evaluation was relatively short and participants spent limited time in VR, making the risk of severe motion sickness low. In comparison to the SSQ, the VRSQ contains fewer items while still providing similar results for oculomotor and disorientation symptoms [6]. The questionnaire uses a 4-point Likert scale ranging from 0 to 3. Participants were instructed to freely explore and test the interactions within each scene until they felt familiar with the experience. After interacting with a scene, participants answered a set of interview questions related to that specific graphical method. The same procedure was then repeated for the second scene. At the end of the session, participants answered additional comparison questions regarding the two graphical methods. Finally, they completed the VRSQ questionnaire [6], concluding the evaluation.

4.4 Data Analysis and Results

Data was collected through semi-structured interviews conducted after each condition and a final comparative session, drawing on audio recordings, transcriptions, and observational notes. The analysis followed an inductive thematic approach, where themes emerged from repeated review of the material rather than a predefined coding scheme. Five themes were identified, organised around the two core dimensions of the research question: visual and physical realism, and user perception.

4.4.1 Theme 1: Visual Realism and Material Association

The two methods produced noticeably different visual impressions. The geometry-based

method was consistently perceived as visually realistic: all four participants independently likened it to a soft, inflatable object, describing it as a balloon or inflatable ball, with one participant drawing a comparison to "Baymax" (Animated character). These associations arose without prompting, suggesting that the deformation reliably communicated the qualities of a soft, compliant material. Participants also noted that the object appeared to bend and vent under pressure rather than squish, which they found visually convincing.

The physics-based method left a different impression. Participants described it as feeling solid and heavy, with comparisons to stone or to the kind of rigid, virtual-looking objects commonly found in games. While this was not considered implausible in itself, it did not align with expectations of soft-body behaviour, and one participant noted that this quality made the object feel more artificial than its counterpart.

4.4.2 Theme 2: Physical Plausibility and Predictability

The geometry-based method was widely regarded as physically plausible and easy to predict. Deformation followed the direction of applied force in a way that felt consistent with how a soft object would behave in the real world (pressing inward or pulling outward produced outcomes that participants found intuitive and legible).

The physics-based method was experienced as less satisfying in this regard. Three out of four participants noted that it responded less than they had anticipated, creating a gap between what they expected to happen and what actually occurred. The method's weight and resistance were acknowledged as realistic for a dense or rigid material. When asked which method they felt more confident predicting, all participants chose the geometry-based approach.

4.4.3 Theme 3: Control and Agency

Participants generally felt more in control when using the geometry-based method, largely because the multiple interaction points allowed them to deform different parts of the surface. This flexibility was seen as a strength. That said, one participant raised an interesting point: although the small interaction spheres gave her a stronger sense of control, they also felt unrealistic, as they allowed her hand to reach further into the object than would be possible in real life. She framed this as a trade-off: removing the spheres would improve realism but reduce the feeling of agency. The physics-based method, by contrast, offered little sense of meaningful influence over the object, which reinforced the impression of limited responsiveness noted above.

4.4.4 Theme 4: Satisfaction and Enjoyment

Both methods were described as enjoyable to interact with, but the geometry-based method consistently received more positive responses. The richer sense of interaction and the material associations participants formed seemed to contribute to a more engaging experience overall. The physics-based method was not described negatively as such, but its limited interactivity made it feel comparatively passive, which dampened the experience. One participant specifically mentioned that haptic feedback and the visual presence of the controller rays added to the feeling of successfully connecting with the spheres.

4.4.5 Theme 5: Perceived Limitations

A few recurring limitations were worth noting. Three out of four participants felt that the effort required to produce a deformation in the geometry-based method was disproportionate to the result. The hand or interaction sphere had to travel further than the deformation seemed to justify, which weakened the

sense of a direct physical connection. Two participants also reported that the lower control points occasionally clipped into the mesh during interaction, making them invisible and harder to use. On the question of elastic recovery, participants' preferences were context-dependent: permanent deformation was seen as appropriate for sculpting purposes, while a return to the original shape was expected in scenarios meant to simulate realistic physical behaviour.

4.4.6 Summary of Interview Results

Across all comparison questions, participants consistently favoured the geometry-based method in terms of visual realism, physical plausibility, predictability, and overall experience. It more convincingly conveyed the impression of a soft, responsive material, while the physics-based method - though not considered implausible - fell short of expectations for soft-body interaction. These findings suggest that geometric deformation approaches such as FFD may be better suited to supporting a convincing and satisfying sense of soft-body interaction in VR, at least compared to the physics-based implementation evaluated here.

5. DISCUSSION

The findings suggest that the geometry-based method was perceived as more realistic, predictable, and enjoyable. However, several limitations make it difficult to draw strong conclusions.

5.1 Comparability of Conditions

The most significant concern is that the two conditions were not equally well-realised. The physics-based method did not produce the level of deformation originally intended, resulting in a less interactive experience than planned. Participants may therefore have preferred the geometry-based method not because geometric deformation is inherently more convincing, but because it was simply more re-

sponsive and engaging to interact with. Additional differences, such as the apparent size and position of the spheres, and the use of coloured balls that may have introduced colour associations, further reduced the comparability of the two conditions. Future studies should ensure the conditions differ only in their underlying deformation method.

It is also worth noting that the distinction between geometric and physics-based deformation is not as clear-cut as the study design implies. Faloutsos, Van De Panne, and Terzopoulos [3] demonstrated that FFD lattices can be augmented with Lagrangian dynamics, effectively combining both approaches. The two methods are therefore better understood as endpoints on a spectrum rather than mutually exclusive alternatives.

5.2 Participant Sample

All four participants were Interactive Media Technology students at KTH, two of whom were concurrently enrolled in the computer graphics course. Their domain knowledge likely shaped both how they interpreted the deformation behaviour and how they articulated their observations, in ways that may not reflect the responses of a general user population. Given these constraints, this evaluation is better understood as a pilot study than a controlled experiment yielding generalisable data. Rather than drawing conclusions from the participant responses themselves, the primary value of this study lies in surfacing methodological considerations for future research. Particularly, the challenges of evaluating graphical techniques in immersive VR, where factors such as haptic feedback, controller visibility, and scene composition all influence perception in ways that are difficult to isolate.

5.3 VR as Evaluation Context

VR introduces additional modalities such as haptics, controller interaction, stereoscopic depth, that contribute to the overall experience and make it difficult to isolate the effect of the deformation method itself. As one participant

noted, haptic feedback and controller visuals played a role in their sense of satisfaction. A more controlled setting might offer a cleaner basis for evaluating visual and physical realism in isolation, though at the cost of ecological validity.

5.4 Perceptual and Technical Trade-offs

The results indicate that the choice of deformation method influences user perception in VR and can meaningfully affect how object interactions are experienced. The findings suggest that FFD may provide a more accessible approach for achieving convincing soft-body interaction in mobile VR environments. This aligns with Faloutsos, Van De Panne, and Terzopoulos [3], who noted that user-defined deformation modes produce predictable, visually convincing results, suggesting that controllability rather than physical accuracy may be the key driver of perceived realism. This is particularly relevant for applications that prioritise artistic control, computational simplicity, or rapid prototyping, such as educational experiences, interactive installations, or lightweight VR applications [1, 12]. At the same time, the findings do not suggest a universally superior method. Instead, each approach appears to present different strengths and trade-offs depending on the intended application and the aspects of realism being prioritised. From an implementation perspective, FFD proved easier to integrate and deploy on the VR headset. Although an existing implementation of PBD from prior work was available [10], substantial modifications were required, including adjustments to the code architecture, shader integration, and interaction handling to support real-time manipulation. These technical challenges are not purely implementation-related, as they may also indirectly affect perceptual outcomes through responsiveness, stability, and interaction quality [7, 10].

5.5 Future Research

The decision to use a sphere as the interaction object appears to have been appropriate, as its simple geometry is well suited to both defor-

mation methods and allows for a more controlled comparison. However, extending the evaluation to more complex geometries could provide additional insights into how different object shapes influence perception and deformation behaviour. Prior work has explored such variations [3, 4] and may serve as a useful reference for future investigations. Future work should prioritise comparable conditions, a larger and more heterogeneous sample, and consideration of whether non-VR evaluation contexts might provide cleaner insight into how deformation methods are perceived on their own merits.

6. CONCLUSION

This study investigated the differences in visual realism, physical plausibility and user perception between Free-Form Deformation and Position-Based Dynamics when applied to interactive object manipulation in VR. Both methods were implemented in Unity and deployed on a Meta Quest 3; each required adaptation for the platform. The implementation process revealed an important asymmetry: FFD could be integrated with relatively little effort, whereas PBD required the re-architecting of file loading for Android, a custom shader, and modifications to support interactivity. The pilot user study favoured FFD across all evaluated dimensions. Participants associated its deformation with soft, compliant materials and found it more predictable and satisfying to interact with. In contrast, PBD was perceived as rigid and less responsive than expected. However, these results must be interpreted with caution, as the PBD implementation did not achieve the intended level of deformation. This means that the comparison is partly between a well-functioning condition and an underperforming one, rather than between two equally realised methods. The primary contribution of this work is therefore a practical account of deploying and comparing two deformation approaches in mobile VR and a set of methodological considerations for future research. A well-controlled comparison between FFD and a fully realised PBD

implementation, evaluated using a larger and more diverse sample, remains a worthwhile and open research question.

References

- [1] Shounak Chatterjee. “Free-Form Shape Modeling in XR: A Systematic Review”. In: *arXiv preprint arXiv:2401.00924* (2024).
- [2] Emergent Mind. *Position-Based Dynamics (PBD)*. 2026. URL: <https://www.emergentmind.com/topics/position-based-dynamics-pbd> (visited on 05/25/2026).
- [3] Petros Faloutsos, Michiel Van De Panne, and Demetri Terzopoulos. “Dynamic free-form deformations for animation synthesis”. In: *IEEE Transactions on Visualization and Computer Graphics* 3.3 (2002), pp. 201–214.
- [4] Sarah FF Gibson and Brian Mirtich. *A survey of deformable modeling in computer graphics*. Tech. rep. Technical report, Mitsubishi Electric Research Laboratories, 1997.
- [5] Robert S Kennedy et al. “Simulator sickness questionnaire: An enhanced method for quantifying simulator sickness”. In: *The international journal of aviation psychology* 3.3 (1993), pp. 203–220.
- [6] Hyun K Kim et al. “Virtual reality sickness questionnaire (VRSQ): Motion sickness measurement index in a virtual reality environment”. In: *Applied ergonomics* 69 (2018), pp. 66–73.
- [7] Taeheon Kim, Jun Ma, and Min Hong. “Real-Time Cloth Simulation in Extended Reality: Comparative Study Between Unity Cloth Model and Position-Based Dynamics Model with GPU”. In: *Applied Sciences* 15.12 (2025), p. 6611.
- [8] Matthias Müller et al. “Position based dynamics”. In: *Journal of Visual Communication and Image Representation* 18.2 (2007), pp. 109–118.
- [9] Thomas W. Sederberg and Scott R. Parry. *Free-Form Deformation of Solid Geometric Models*. Tech. rep. 4. Brigham Young University, 1986. DOI: 10.1145/15886.15903.
- [10] Nak-Jun Sung et al. “Real-Time Physics Simulation Method for XR Application”. In: *Computers* 14.1 (2025), p. 17.
- [11] TutorialsPoint. *Computer Graphics - Deformation Animation*. Accessed: 2026-04-23. n.d.
- [12] Miklos Vincze, Bela Molnar, and Miklos Kozlovszky. “3D visualization in digital medicine using XR technology”. In: *Future Internet* 15.9 (2023), p. 284.